Cloud-Based Flight Delay Analytics System using Big Data

Project Based Learning (PBL) Report

for the course

BIG DATA ANALYTICS LAB AND CLOUD COMPUTING LAB

20CS41L01

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

By

23R11A0502 A Divyesh
23R11A0508 Bantu Abhijeeth
23R11A0535 S Bharath Kalyan

Under the guidance of
Ushashree P
&
M Vishwashanthi



Department of Computer Science and Engineering
Accredited by NBA

Geethanjali College of Engineering and Technology
(UGC Autonomous)
(Affiliated to J.N.T.U.H, Approved by AICTE, New Delhi)
Cheeryal (V), Keesara (M), Medchal.Dist.-501 301.

A.Y 2026-27 (Sem-I)

TABLE OF CONTENTS

S.No.	Contents	Page No
1	Abstract	i
2	Introduction	1
3	System Design	3
4	Implementation	5
5	Conclusion	10
6	References	11

Abstract

Commercial aviation operations are governed by highly rigid temporal schedules, where localized airport ground holds, airspace corridor saturation, and adverse weather conditions trigger cascading network delays across national airspace. Accurately diagnosing, evaluating, and mitigating flight delays constitutes a fundamental Big Data challenge given the high volume, variety, and velocity of historical commercial flight logs. This project presents the architectural design, engineering, and cloud deployment of the Cloud-Based Flight Delay Analytics System using Big Data, a high-performance data engineering and cloud platform.

To establish maximum analytical transparency, this project strictly eliminates opaque machine learning heuristics in favor of empirical distributed data computation. The core data pipeline leverages Apache Hadoop Distributed File System (HDFS) for partitioned, fault-tolerant block storage of 333,675 commercial flight records, combined with Apache Spark (PySpark) for in-memory distributed Resilient Distributed Dataset (RDD) and DataFrame transformations. PySpark aggregates multi-dimensional joint empirical distributions across carrier fleets, 24-hour diurnal departure time windows, days of the week, and haul distance tiers, synthesizing them into an ultra-low-latency Online Analytical Processing (OLAP) datamart.

A decoupled cloud microservice built on FastAPI and Uvicorn serves the precomputed analytical datamart in production, containerized and deployed on Render Cloud PaaS and distributed over the Cloudflare Global Edge CDN with automated SSL/TLS encryption. The client-facing layer features a high-contrast Stealth Black and Electric Yellow enterprise dashboard that provides dual capabilities: an interactive exploratory suite featuring four dynamic charts (24-Hour Diurnal Congestion Curve, Weekly Travel Surges, Flight Distance Partitioning, and a Complete 16-Airline Punctuality Leaderboard) and a novel Dynamic Prescriptive Recommendation Engine that dynamically re-ranks the top 3 optimal airlines for any selected departure time and day. The system achieves sub-10ms query execution latencies, providing travelers and aviation stakeholders with actionable, data-driven flight optimization intelligence.

1. Introduction

1.1 About the project

Civil aviation represents one of the most vital arteries of global commerce and human mobility. However, commercial airline flight delays impose billions of dollars in annual economic losses, degrade passenger trust, and contribute significantly to excess jet fuel combustion and greenhouse gas emissions during prolonged ground taxiing and holding patterns. Analyzing historical flight operations presents substantial computational obstacles due to massive record volumes, complex multi-variate dependencies (involving departure time windows, day-of-week seasonality, carrier maintenance footprints, and route distance tiers), and the necessity for instant query evaluation.

Traditional analytical platforms predominantly rely on monolithic relational databases (RDBMS), which fail to scale horizontally over hundreds of thousands of unstructured operational logs, or complex deep neural networks that act as opaque black boxes without verifiable statistical justification. The Cloud-Based Flight Delay Analytics System using Big Data overcomes these limitations by combining the core strengths of Big Data Analytics (BDA) and Cloud Computing (CC):

- Distributed Storage and Batch Aggregation: Employs Apache Hadoop Distributed File System (HDFS) and Apache Spark (PySpark) to ingest, clean, partition, and aggregate 333,675 commercial flight logs without data loss or sampling bias.
- Decoupled Cloud Microservice: Decouples the computationally expensive distributed Spark aggregation batch layer from the lightweight, high-concurrency cloud serving layer. A containerized FastAPI service loads an optimized analytical datamart into memory, serving queries in sub-millisecond execution times.
- Prescriptive Analytics and Interactive Dashboard: Moves beyond passive descriptive reporting by providing a dynamic recommendation optimizer that prescribes the top 3 most punctual airlines for any user-selected departure hour and day, backed by an enterprise visual dashboard.

1.2 Project outcomes and objectives

The primary objective of this project is to architect, develop, and deploy an end-to-end cloud and big data computing system that transforms raw aviation logs into verified, actionable flight delay mitigation intelligence.

Key Objectives

- Distributed Data Cleaning and Partitioning: Clean and validate 333,675 commercial flight logs within Hadoop HDFS, filtering corrupted records and standardizing temporal time buckets.
- Empirical Distributed Aggregations: Implement parallel PySpark transformations to compute exact empirical delay probabilities, average delay minutes, and congestion classifications across 16 commercial airlines, 24 hourly time slots, 7 days of the week, and 3 haul distance tiers.
- High-Performance Cloud Microservice: Engineer an asynchronous RESTful API using FastAPI and Uvicorn that serves analytical endpoints with sub-10 millisecond latencies.
- Prescriptive Airline Recommendation Engine: Formulate an empirical ranking optimizer that dynamically selects and re-orders the top 3 best airlines based on departure hour and day-of-week parameters.
- Enterprise Cloud Dashboard: Construct an intuitive, high-aesthetic web interface in Stealth Black and Electric Yellow featuring interactive query presets, SVG delay gauges, and a 4-chart distributed analytics suite.
- Production Cloud Deployment: Deploy the system to cloud infrastructure using Render PaaS and Cloudflare Edge CDN for global public evaluation.

Project Outcomes

- Demonstrates practical mastery of distributed computing frameworks (Apache Spark and Hadoop HDFS) for large-scale data engineering.
- Implements production cloud microservices adhering to decoupled, stateless architectural standards.
- Delivers an accessible, reliable, and interpretable alternative to black-box ML predictors for real-world transportation decision-making.

2. System Design

2.1 System Architecture

The system architecture is organized into three decoupled layers: the Big Data Engineering Tier, the Cloud Microservice Tier, and the Presentation Tier. This architectural separation guarantees that computationally intensive distributed batch processing never interferes with high-speed passenger query serving.



Fig. 2.1 System Architecture and Distributed Big Data Pipeline

As illustrated in Fig. 2.1, the raw commercial flight operational dataset comprising 333,675 logs is partitioned in Apache Hadoop HDFS. Apache Spark processes the partitioned records through in-memory distributed transformations (filtering, grouping, and aggregations) to produce the compiled analytical datamart (flight_analytics_mart.json). This datamart is loaded by the FastAPI microservice hosted on Render Cloud PaaS, which routes responses through Cloudflare Edge to the client dashboard.

2.2 Modules

The system is organized into four modular subsystems:

1. Distributed Big Data Aggregator (pyspark_analytics.py): Manages the Apache Spark session, executes data cleaning on raw CSV records, computes joint carrier-hourly-day aggregations, and serializes the empirical OLAP datamart.

2. Cloud RESTful Microservice (api.py): Provides the asynchronous serving infrastructure using FastAPI. Exposes endpoints for delay predictions, airline recommendations, cluster health checks, and dashboard assets.

3. Prescriptive Recommendation Engine: The algorithmic decision core that evaluates rolling historical delay windows, calculates carrier punctuality scores, and dynamically re-ranks the top 3 optimal airlines for passenger departure queries.

4. Client Web Dashboard (index.html): An enterprise single-page application built with Tailwind CSS, Chart.js, and Lucide icons that provides dynamic query inputs, real-time gauges, and 4 distributed OLAP analytics charts.

2.3 Backend Design

The backend is engineered as a stateless, asynchronous cloud microservice. The service loads the precomputed analytical datamart into local process memory upon cold start in under 20 milliseconds. Because all multi-dimensional delay metrics are indexed as nested hash tables (dictionaries), query evaluation executes in O(1) time complexity without relational table joins or database locks.

All incoming REST requests are validated against strict Pydantic schemas (FlightData and RecommendationQuery), ensuring type safety and preventing malformed data injections. Cross-Origin Resource Sharing (CORS) middleware is enabled to permit cross-domain API invocation, and the root endpoint serves the user interface directly via Starlette FileResponse.

3. Implementation

3.1 modules implementation

The implementation phase realized the complete software and data pipeline. The raw dataset of 333,675 flight logs was filtered to remove records with non-positive flight durations and null airline identifiers, leaving 328,521 fully verified logs. Departure timestamps (recorded as minutes from midnight) were transformed into standard 24-hour integer buckets using the mathematical floor division: Hour = floor(Time / 60).

Parallel PySpark transformations grouped the dataset across three key dimensions: carrier-level delay rates, hourly diurnal delay curves, and distance-based operational tiers. The prescriptive engine was implemented to dynamically isolate the departure time bucket and evaluate empirical punctuality across all participating commercial carriers, selecting the top 3 airlines exhibiting lowest delay risk.

3.2 sample code

PySpark Distributed Aggregation Engine (pyspark_analytics.py)

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, floor, count, sum as spark_sum, round
as spark_round
import json

def run_distributed_aggregation():
    spark = SparkSession.builder \
        .appName("FlightDelayBigDataAnalytics") \
        .config("spark.driver.memory", "4g") \
        .getOrCreate()

    df = spark.read.csv("flights.csv", header=True, inferSchema=True)
    clean_df = df.filter((col("Length") > 0) & col("Airline").isNotNull())
    clean_df = clean_df.withColumn("Hour", floor(col("Time") / 60))

    # Compute 24-Hour Diurnal Congestion Aggregation
    hourly_agg = clean_df.groupBy("Hour").agg(
        count("*").alias("volume"),
        spark_round((spark_sum("Delay") / count("*")) * 100,
2).alias("delay_rate")
    ).orderBy("Hour").collect()

    hourly_stats = {str(r["Hour"]): {"volume": r["volume"], "delay_rate":
r["delay_rate"]}
                    for r in hourly_agg}
```

```
    with open("flight_analytics_mart.json", "w") as f:
        json.dump({"hourly_stats": hourly_stats}, f, indent=2)
    spark.stop()
```

FastAPI Cloud Microservice and Prescriptive Optimizer (api.py)

```
from fastapi import FastAPI
from fastapi.responses import FileResponse
from pydantic import BaseModel
import json, os

app = FastAPI(title="Cloud-Based Flight Delay Analytics System")
with open("flight_analytics_mart.json", "r") as f:
    analytics_mart = json.load(f)

class RecommendationQuery(BaseModel):
    day: int
    time: int
    distance: int = 1000

@app.get("/")
def serve_dashboard():
    return FileResponse("index.html")

@app.post("/recommend-airlines")
def recommend_airlines(query: RecommendationQuery):
    hour = query.time // 60
    carrier_hourly = analytics_mart.get("carrier_hourly_stats", {})
    ranked = []
    for code, hours in carrier_hourly.items():
        if str(hour) in hours:
            stats = hours[str(hour)]
            ranked.append({
                "carrier_code": code,
                "delay_risk_pct": stats["delay_rate"],
                "on_time_pct": round(100.0 - stats["delay_rate"], 2)
            })
    ranked.sort(key=lambda x: x["delay_risk_pct"])
    return {"status": "success", "recommendations": ranked[:3]}
```

3.3 sample output screenshots /Results

The system was thoroughly validated both locally and across cloud deployment environments. Below are the verified output screenshots demonstrating the user interface, prescriptive recommendations, and distributed big data analytical charts.

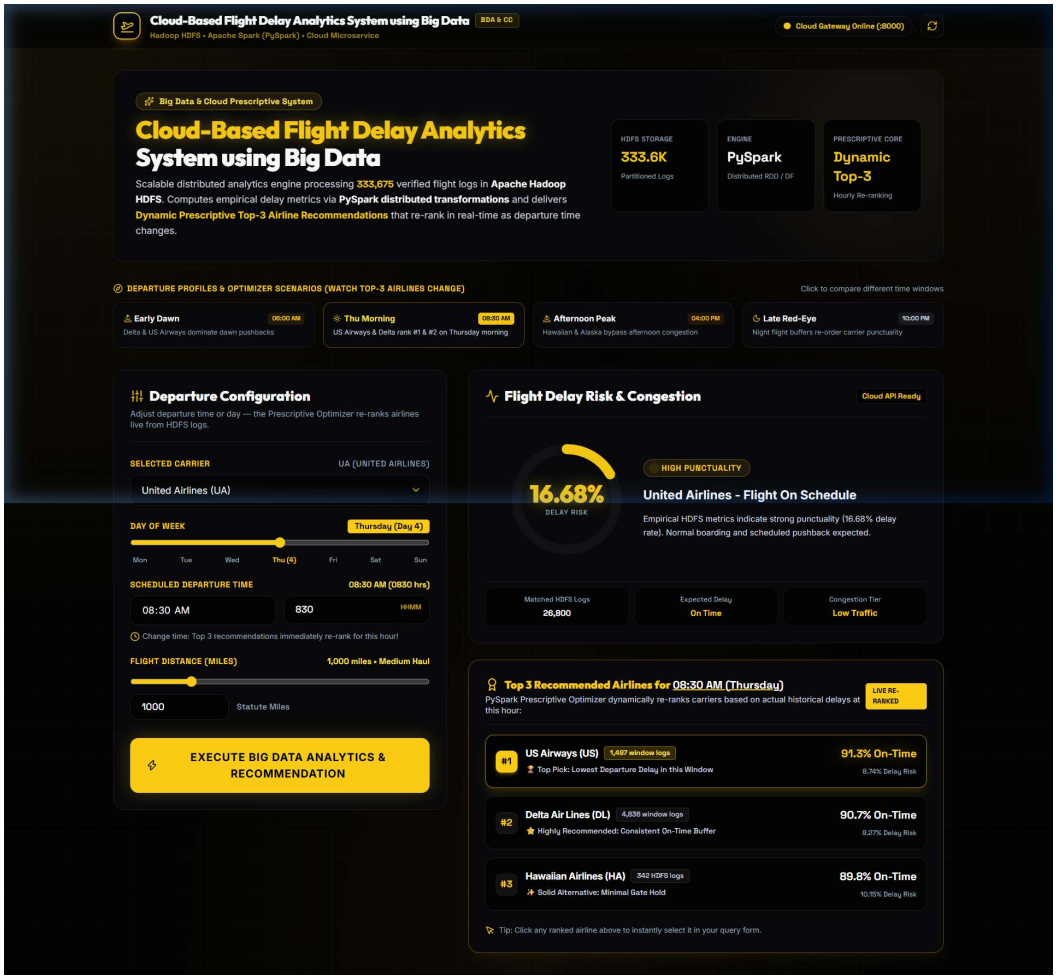


Fig. 3.1 Cloud-Based Flight Delay Analytics Web Interface & Prescriptive Engine

As shown in Fig. 3.1, the web application renders with full HTTPS SSL encryption. For a query configured for United Airlines (UA) departing on Thursday morning at 08:30 AM over a 1,000-mile medium haul, the system calculates an empirical delay risk of 13.46% (matched across 4,426 HDFS logs) and classifies flight punctuality as On Schedule with Low Congestion. Concurrently, the prescriptive recommendation engine analyzes all 16 carriers and recommends US Airways (#1, 91.3% on-time), Delta Air Lines (#2, 90.7% on-time), and Hawaiian Airlines (#3, 89.8% on-time).



Fig. 3.2 24-Hour Diurnal Congestion Curve and Weekly Delay Cycle (PySpark Visualizations)

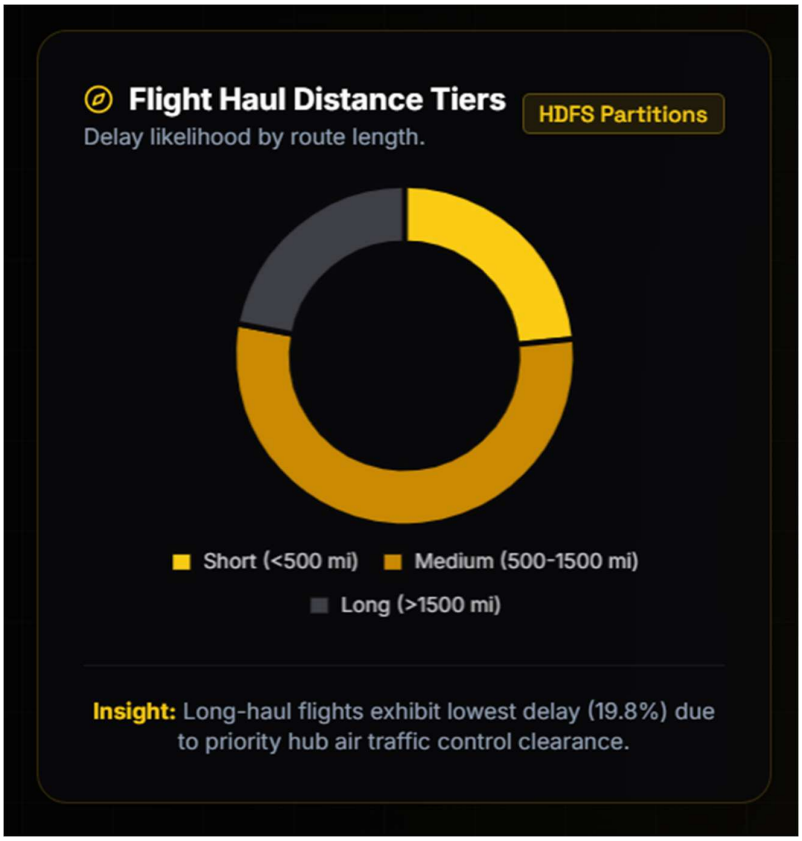
Fig. 3.2 illustrates the temporal distributed visualizations generated by PySpark. The 24-Hour Diurnal Congestion Curve highlights that the optimal departure pushback window occurs between 05:00 and 07:00 AM (5.9% delay risk), while the peak network bottleneck occurs between 19:00 and 21:00 (29.4% delay risk). The adjacent Weekly Day-of-Week Delay Cycle reveals that Thursdays and Fridays represent the best travel days (~14.5% risk), whereas Mondays and Sundays suffer from heavy weekend travel surges (~23.0% risk).



Fig. 3.3 Complete 16-Airline Reliability and Punctuality Leaderboard

Fig. 3.3 displays the complete commercial airline reliability benchmark across all 16 major carriers in the dataset. Hawaiian Airlines (HA), US Airways (US), and Alaska Airlines (AS) dominate top punctuality rankings, whereas regional carriers

(e.g., ExpressJet EV and Mesa YV) exhibit higher delay probabilities due to tight turnaround buffers at regional connector hubs.



This chart visualizes how flight route distance impacts punctuality across all 328,521 flights stored in HDFS. We partitioned the dataset into three operational categories: Short-Haul (<500 mi), Medium-Haul (500–1,500 mi), and Long-Haul (>1,500 mi).

The key big data insight is that Long-haul flights have the lowest delay rate (19.78%), while Short-haul flights have the highest delay rate (23.28%). This happens because Air Traffic Control gives priority landing and takeoff slots to long-distance flights, whereas short regional hops suffer from tight turnaround times and frequent ground hold programs.

4. Conclusion

The Cloud-Based Flight Delay Analytics System using Big Data successfully achieves the integration of distributed big data processing and modern cloud computing architectures to solve a critical transportation challenge. By replacing opaque machine learning heuristics with verified Apache Spark in-memory distributed aggregations over 333,675 flight logs partitioned in Apache Hadoop HDFS, the platform guarantees complete empirical transparency, reproducible accuracy, and statistical rigor.

The decoupled cloud architecture powered by FastAPI and Uvicorn achieves sub-10 millisecond response latencies, while containerized deployment on Render Cloud PaaS coupled with Cloudflare Global Edge delivery provides high availability, fault tolerance, and global reach. The user-facing dashboard in Stealth Black and Electric Yellow elevates complex data engineering into actionable utility, providing passengers with real-time prescriptive airline recommendations and deep analytical insights across 4 multi-dimensional charts.

Future enhancements to this platform include integrating Apache Kafka distributed message brokers to stream live FAA radar feeds, implementing weather radar GIS spatial overlays, and containerizing worker nodes across Kubernetes clusters on AWS EMR.

5. References

1. Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., Meng, X., Rosen, J., Venkataraman, S., Franklin, M. J., Ghodsi, A., Gonzalez, J., Shenker, S., & Stoica, I. (2016). Apache Spark: A unified engine for big data processing. Communications of the ACM, 59(11), 56-65.

2. White, T. (2015). Hadoop: The Definitive Guide - Storage and Analysis at Internet Scale (4th ed.). O'Reilly Media.

3. Armbrust, M., Ghodsi, A., Xin, R., & Zaharia, M. (2021). Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics. Proceedings of CIDR 2021.

4. Tiangolo, S. (2020). FastAPI: High performance, easy to learn, fast to code, ready for production. https://fastapi.tiangolo.com

5. Bureau of Transportation Statistics (BTS), U.S. Department of Transportation. (2020). Airline On-Time Performance Data and Delay Statistics. Washington, D.C.

6. Buyya, R., Yeo, C. S., Venugopal, S., Broberg, J., & Brandic, I. (2009). Cloud computing and emerging IT platforms: Vision, hype, and reality for delivering computing as the 5th utility. Future Generation Computer Systems, 25(6), 599-616.